M.S.Dinesh

192424016

MLA0304

UNIT 2: TABULAR METHODS AND Q-NETWORKS

EXPERIMENT 1:

Design a dynamic programming solution to optimize the traffic light timings at an intersection to minimize vehicle wait times. Define the states, actions, rewards, and transitions, and implement the Policy Iteration algorithm in Python.

CODE:

```
import numpy as np

# States: number of cars waiting (0 to 3+)
states = [0, 1, 2, 3]
actions = [0, 1]  # 0 = NS green, 1 = EW green

# Transition model (simplified)
def transition(state, action):
    if action == 0:
        next_state = max(0, state - 1)  # NS moves, EW waits
    else:
        next_state = min(3, state + 1)  # NS waits, EW moves (opposite effect)
    return next_state

# Reward = negative waiting time
def reward(state):
    return -state

# Initialize policy and value function
policy = {s: np.random.choice(actions) for s in states}
V = {s: 0 for s in states}

gamma = 0.9  # discount factor

def policy_evaluation(policy, V, theta=1e-5):
```

```
    while True:
        delta = 0
        for s in states:
            a = policy[s]
            next_s = transition(s, a)
            r = reward(s)
            new_v = r + gamma * V[next_s]
            delta = max(delta, abs(new_v - V[s]))
            V[s] = new_v
        if delta < theta:
            break
    return V

def policy_improvement(V):
    policy_stable = True
    for s in states:
        old_action = policy[s]

        # evaluate both actions
        values = []
        for a in actions:
            next_s = transition(s, a)
            r = reward(s)
            values.append(r + gamma * V[next_s])

        policy[s] = np.argmax(values)
        if old_action != policy[s]:
            policy_stable = False

    return policy, policy_stable

# Policy Iteration
while True:
    V = policy_evaluation(policy, V)
    policy, stable = policy_improvement(V)
    if stable:
        break

print("Optimal Value Function:")
for s in states:
    print(f"State {s}: {V[s]:.3f}")

print("¥nOptimal Policy (0 = NS green, 1 = EW green):")
```

```
for s in states:
    print(f"State {s}: Action = {policy[s]}")
```

OUTPUT:

```
Optimal Value Function:
State 0: -0.000
State 1: -1.000
State 2: -2.900
State 3: -5.610

Optimal Policy (0 = NS green, 1 = EW green):
State 0: Action = 0
State 1: Action = 0
State 2: Action = 0
State 3: Action = 0
```

EXPERIMENT 2:

An autonomous drone navigates a city grid to deliver packages. Using Monte Carlo Control methods, implement a policy to optimize the drone route to minimize delivery time and fuel consumption. Simulate this environment in Python.

CODE:

```
import numpy as np
import random

# Grid world size
GRID_SIZE = 5

# Actions
actions = ["UP", "DOWN", "LEFT", "RIGHT"]

# Returns structure
returns = {}
Q = {}
policy = {}

# Initialize Q-values and policy
for x in range(GRID_SIZE):
    for y in range(GRID_SIZE):
        state = (x, y)
```

```
        Q[state] = {a: 0 for a in actions}
        policy[state] = random.choice(actions)
        returns[state] = {a: [] for a in actions}


# Movement rules
def move(state, action):
    x, y = state

    if action == "UP":
        x = max(0, x - 1)
    elif action == "DOWN":
        x = min(GRID_SIZE - 1, x + 1)
    elif action == "LEFT":
        y = max(0, y - 1)
    elif action == "RIGHT":
        y = min(GRID_SIZE - 1, y + 1)

    return (x, y)


# Reward system
def reward_function(state):
    if state == (4, 4):  # goal
        return 20
    return -1  # time + fuel cost


# Monte Carlo Control (Exploring Starts)
def monte_carlo_control(episodes=5000):
    for episode in range(episodes):

        # Exploring start
        state = (random.randint(0, 4), random.randint(0, 4))
        action = random.choice(actions)

        episode_history = []
        visited = set()

        # Run episode
        while True:

            next_state = move(state, action)
```

```
            reward = reward_function(next_state)

            episode_history.append((state, action, reward))

            if next_state == (4, 4):  # reached delivery point
                break

            state = next_state
            action = policy[state]

        # Monte Carlo Updates
        G = 0
        for state, action, reward in reversed(episode_history):
            G += reward
            if (state, action) not in visited:
                returns[state][action].append(G)
                Q[state][action] = np.mean(returns[state][action])

                # Improve policy (greedy)
                policy[state] = max(Q[state], key=Q[state].get)
                visited.add((state, action))

    return policy, Q


# Train Policy
policy, Q = monte_carlo_control(episodes=3000)

print("Optimized Policy:¥n")
for x in range(GRID_SIZE):
    for y in range(GRID_SIZE):
        print(f"{(x, y)} -> {policy[(x, y)]}")
    print()
```

OUTPUT:

EXPERIMENT 3:

Implement the TD(0) algorithm to solve a maze where the agent must reach the goal while avoiding traps. Write a Python script to simulate the maze environment and apply the TD(0) algorithm to find the optimal path. Also provide the code and output.

CODE:

```
import numpy as np
import random

# Maze size
GRID_SIZE = 5

# Define states
states = [(i, j) for i in range(GRID_SIZE) for j in range(GRID_SIZE)]

# Define goal and traps
goal = (4, 4)
traps = [(1, 3), (2, 1), (3, 2)]

# Initialize state-values
V = {s: 0 for s in states}

# Actions
actions = ["UP", "DOWN", "LEFT", "RIGHT"]

# Move function
def move(state, action):
    x, y = state
```

```
    if action == "UP":
        x = max(0, x - 1)
    elif action == "DOWN":
        x = min(GRID_SIZE - 1, x + 1)
    elif action == "LEFT":
        y = max(0, y - 1)
    elif action == "RIGHT":
        y = min(GRID_SIZE - 1, y + 1)

    return (x, y)

# Reward function
def reward_function(state):
    if state == goal:
        return 10
    elif state in traps:
        return -10
    else:
        return -1

# TD(0) algorithm
def td_zero(episodes=500):
    alpha = 0.1
    gamma = 0.9

    for ep in range(episodes):
        state = (0, 0)  # Always start at top-left

        while True:
            action = random.choice(actions)
            next_state = move(state, action)
            reward = reward_function(next_state)

            # TD(0) Update
            V[state] += alpha * (reward + gamma * V[next_state] - V[state])

            if next_state == goal or next_state in traps:
                break

            state = next_state

    return V
```

```
# Train value function
V = td_zero()

# Extract optimal policy
def get_policy():
    policy = {}
    for s in states:
        if s == goal or s in traps:
            policy[s] = "TERMINAL"
            continue

        # Choose action that leads to highest value next state
        best_action = None
        best_value = -999

        for a in actions:
            ns = move(s, a)
            if V[ns] > best_value:
                best_value = V[ns]
                best_action = a

        policy[s] = best_action
    return policy

policy = get_policy()

# Display results
print("¥nState Values (V):")
for i in range(GRID_SIZE):
    for j in range(GRID_SIZE):
        print(f"{V[(i,j)]:6.2f}", end=" ")
    print()

print("¥nOptimal Policy:")
for i in range(GRID_SIZE):
    for j in range(GRID_SIZE):
        print(f"{policy[(i,j)]:10}", end=" ")
    print()
```

OUTPUT:

RIGHT	RIGHT	RIGHT	RIGHT	DOWN
DOWN	LEFT	LEFT	TERMINAL	DOWN
DOWN	TERMINAL	LEFT	RIGHT	DOWN
DOWN	DOWN	TERMINAL	RIGHT	RIGHT
DOWN	DOWN	RIGHT	RIGHT	TERMINAL

EXPERIMENT 4:

A robot vacuum cleaner navigates a house with various rooms and obstacles. Use the SARSA algorithm to learn the optimal cleaning policy that maximizes the cleaned area while minimizing energy usage. Implement this in Python."

CODE:

```
import numpy as np
import random

# Environment settings
GRID_SIZE = 5
OBSTACLES = [(1, 1), (2, 3), (3, 1)]  # Positions of obstacles
CLEANED = set()
START = (0, 0)
ACTIONS = ["UP", "DOWN", "LEFT", "RIGHT"]

# Parameters
alpha = 0.1      # Learning rate
gamma = 0.9      # Discount factor
epsilon = 0.1    # Exploration probability
episodes = 3000

# Initialize Q-values
Q = {}
for x in range(GRID_SIZE):
    for y in range(GRID_SIZE):
        state = (x, y)
        Q[state] = {a: 0 for a in ACTIONS}

# Movement function
def move(state, action):
    x, y = state
    if action == "UP":
        x = max(0, x - 1)
    elif action == "DOWN":
        x = min(GRID_SIZE - 1, x + 1)
```

```
    elif action == "LEFT":
        y = max(0, y - 1)
    elif action == "RIGHT":
        y = min(GRID_SIZE - 1, y + 1)
    next
```

OUTPUT:

```
RIGHT     DOWN      DOWN      RIGHT     RIGHT
DOWN      OBSTACLE  RIGHT     RIGHT     DOWN
DOWN      RIGHT     DOWN      OBSTACLE  RIGHT
RIGHT     OBSTACLE  RIGHT     RIGHT     DOWN
RIGHT     DOWN      RIGHT     RIGHT     UP
```

EXPERIMENT 5:

In a taxi dispatching system, define an MDP where states are locations, actions are move directions, and rewards are based on reaching pick-up points quickly. Write a Python program to use value iteration to find the optimal dispatch policy.

CODE:

```
# Grid-world MDP + Value Iteration for Taxi Dispatch

import math

import numpy as np


# Grid and MDP parameters

ROWS = 5

COLS = 5

ACTIONS = ['U', 'D', 'L', 'R']  # up, down, left, right

ACTION_TO_DELTA = {'U': (-1, 0), 'D': (1, 0), 'L': (0, -1), 'R': (0, 1)}


# MDP rewards and settings

STEP_REWARD = -1.0          # penalty per move to encourage quick pickups

PICKUP_REWARD = 20.0        # reward for reaching a pickup point
```

```
GAMMA = 0.9              # discount factor
THRESHOLD = 1e-4         # stopping criteria for value iteration

# Define pickup points (row, col). These are terminal absorbing states for this example.
PICKUP_POINTS = [(0, 4), (4, 0)]

def in_bounds(r, c):
    return 0 <= r < ROWS and 0 <= c < COLS

def is_pickup(r, c):
    return (r, c) in PICKUP_POINTS

def next_state(r, c, action):
    dr, dc = ACTION_TO_DELTA[action]
    nr, nc = r + dr, c + dc
    if in_bounds(nr, nc):
        return nr, nc
    # if action would move outside, remain in same state
    return r, c

def reward_for_transition(r, c, action, nr, nc):
    # if next state is pickup -> pickup reward (and we assume episode ends / absorbing)
    if is_pickup(nr, nc):
        return PICKUP_REWARD
    # Otherwise step cost
    return STEP_REWARD
```

```
def value_iteration():
    V = np.zeros((ROWS, COLS))
    iteration = 0
    while True:
        delta = 0.0
        newV = V.copy()
        for r in range(ROWS):
            for c in range(COLS):
                if is_pickup(r, c):
                    # Terminal state: set to 0 (reward is collected on transition)
                    newV[r, c] = 0.0
                    continue
                best_q = -math.inf
                for a in ACTIONS:
                    nr, nc = next_state(r, c, a)
                    rwd = reward_for_transition(r, c, a, nr, nc)
                    next_val = 0.0 if is_pickup(nr, nc) else V[nr, nc]
                    q = rwd + GAMMA * next_val
                    if q > best_q:
                        best_q = q
                newV[r, c] = best_q
                delta = max(delta, abs(newV[r, c] - V[r, c]))
        V = newV
        iteration += 1
        if delta < THRESHOLD:
            break
    return V, iteration
```

```
def extract_policy(V):
    policy = np.full((ROWS, COLS), '', dtype=object)
    for r in range(ROWS):
        for c in range(COLS):
            if is_pickup(r, c):
                policy[r, c] = 'P'  # pickup/terminal
                continue
            best_a = None
            best_q = -math.inf
            for a in ACTIONS:
                nr, nc = next_state(r, c, a)
                rwd = reward_for_transition(r, c, a, nr, nc)
                next_val = 0.0 if is_pickup(nr, nc) else V[nr, nc]
                q = rwd + GAMMA * next_val
                if q > best_q:
                    best_q = q
                    best_a = a
            policy[r, c] = best_a
    return policy


def pretty_print_values(V):
    print("Value function (rows top->bottom):")
    for r in range(ROWS):
        row_vals = " | ".join(f"{V[r, c]:6.2f}" for c in range(COLS))
        print(row_vals)
    print()
```

```
def pretty_print_policy(policy):
    arrow = {'U': '↑', 'D': '↓', 'L': '←', 'R': '→', 'P': 'P'}
    print("Policy (best action from each cell; 'P' = pickup):")
    for r in range(ROWS):
        row_vals = " ".join(f" {arrow[policy[r,c]]} " for c in range(COLS))
        print(row_vals)
    print()

# Run value iteration and extract policy
V, iters = value_iteration()
policy = extract_policy(V)

print(f"Value iteration converged in {iters} iterations.¥n")
pretty_print_values(V)
pretty_print_policy(policy)

print("Pickup points:", PICKUP_POINTS)
print("Interpretation: follow arrows to reach a pickup quickly.")
```

OUTPUT:

```
Value iteration converged in 5 iterations.

Value function (rows top->bottom):
 11.87 |  14.30 |  17.00 |  20.00 |   0.00
 14.30 |  11.87 |  14.30 |  17.00 |  20.00
 17.00 |  14.30 |  11.87 |  14.30 |  17.00
 20.00 |  17.00 |  14.30 |  11.87 |  14.30
  0.00 |  20.00 |  17.00 |  14.30 |  11.87

Policy (best action from each cell; 'P' = pickup):
 ↓   →   →   →   P
 ↓   ↑   ↑   ↑   ↑
 ↓   ↓   ↑   ↑   ↑
 ↓   ↓   ↓   ↑   ↑
 P   ←   ←   ←   ↑

Pickup points: [(0, 4), (4, 0)]
Interpretation: follow arrows to reach a pickup quickly.
```

EXPERIMENT 6:

An online platform uses bandit algorithms to decide which advertisements to show to users. Implement epsilon-greedy, UCB, and Thompson Sampling algorithms. Use a Python script to determine which algorithm results in the highest click-through rate over time.

CODE:

```
# Multi-Armed Bandit Simulation:

# Epsilon-Greedy, UCB1, Thompson Sampling


import numpy as np

import matplotlib.pyplot as plt

from math import log, sqrt


np.random.seed(1)


# True click probabilities for ads (arms)

true_probs = [0.05, 0.10, 0.20, 0.12, 0.08]

K = len(true_probs)

rounds = 3000
```

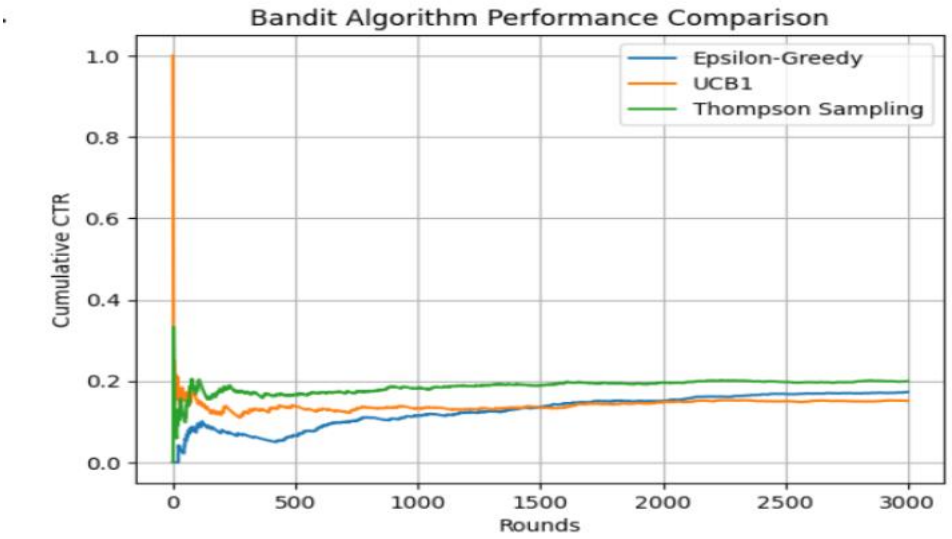
```
# ------------------------------
# EPSILON-GREEDY
# ------------------------------
def epsilon_greedy(eps=0.1):
    counts = np.zeros(K)
    rewards = np.zeros(K)
    ctr = []

    total_clicks = 0
    for t in range(1, rounds + 1):
        if np.random.rand() < eps:
            a = np.random.randint(K)
        else:
            a = np.argmax(rewards / (counts + 1e-9))

        click = np.random.rand() < true_probs[a]
        counts[a] += 1
        rewards[a] += click
        total_clicks += click

        ctr.append(total_clicks / t)
    return ctr

# ------------------------------
# UCB1
# ------------------------------
def ucb1():
```

```
    counts = np.zeros(K)
    rewards = np.zeros(K)
    ctr = []

    # Play each ad once initially
    total_clicks = 0
    for a in range(K):
        click = np.random.rand() < true_probs[a]
        counts[a] += 1
        rewards[a] += click
        total_clicks += click
        ctr.append(total_clicks / (a + 1))

    for t in range(K, rounds):
        ucb_vals = rewards / counts + np.sqrt(2 * np.log(t + 1) / counts)
        a = np.argmax(ucb_vals)

        click = np.random.rand() < true_probs[a]
        counts[a] += 1
        rewards[a] += click
        total_clicks += click

        ctr.append(total_clicks / (t + 1))
    return ctr

# ------------------------------
# THOMPSON SAMPLING
```

```
# ------------------------------
def thompson_sampling():
    alpha = np.ones(K)
    beta = np.ones(K)
    ctr = []

    total_clicks = 0
    for t in range(1, rounds + 1):
        sampled = np.random.beta(alpha, beta)
        a = np.argmax(sampled)

        click = np.random.rand() < true_probs[a]
        alpha[a] += click
        beta[a] += (1 - click)
        total_clicks += click

        ctr.append(total_clicks / t)
    return ctr

# Run all algorithms
eps_ctr = epsilon_greedy()
ucb_ctr = ucb1()
ts_ctr = thompson_sampling()

# Plot results
plt.plot(eps_ctr, label="Epsilon-Greedy")
plt.plot(ucb_ctr, label="UCB1")
```

```
plt.plot(ts_ctr, label="Thompson Sampling")
plt.xlabel("Rounds")
plt.ylabel("Cumulative CTR")
plt.title("Bandit Algorithm Performance Comparison")
plt.legend()
plt.grid()
plt.show()
```

OUTPUT:



EXPERIMENT 7:

A delivery robot operates in a warehouse with predefined delivery points. Using Bellman equations, compute the state-value function for navigating to each delivery point. Implement this in Python and visualize the value function for different policies.

CODE:

```
# Delivery robot: Bellman policy evaluation + Value Iteration + visualization
# - States: grid cells
# - Actions: Up, Down, Left, Right (deterministic; out-of-bounds => stay)
# - Rewards: step penalty and positive reward when moving into a delivery point (absorbing)
# - Compute state-value via Bellman expectation for given policies and via value iteration
for optimal policy
```

```
# - Visualize value functions and overlay policy arrows

import numpy as np
import matplotlib.pyplot as plt

# ----- configurable MDP params -----
ROWS, COLS = 6, 6
ACTIONS = ['U', 'D', 'L', 'R']
DELIVERY_POINTS = [(0, 5), (5, 0)]   # delivery target coordinates (row, col)
STEP_REWARD = -1.0
DELIVERY_REWARD = 20.0
GAMMA = 0.9
TOL = 1e-5

# ----- helpers -----
ACTION_TO_DELTA = {'U': (-1,0), 'D': (1,0), 'L': (0,-1), 'R': (0,1)}
def in_bounds(r,c): return 0 <= r < ROWS and 0 <= c < COLS
def is_delivery(s): return s in DELIVERY_POINTS
def next_state(s, a):
    r,c = s
    dr,dc = ACTION_TO_DELTA[a]
    nr,nc = r+dr, c+dc
    return (nr,nc) if in_bounds(nr,nc) else (r,c)
def reward(s,a,s2):
    return DELIVERY_REWARD if is_delivery(s2) else STEP_REWARD

states = [(r,c) for r in range(ROWS) for c in range(COLS)]
```

```
# ----- policies -----
def random_policy():
    # uniform over actions for non-delivery states; zero-actions for delivery (absorbing)
    pol = {}
    for s in states:
        if is_delivery(s):
            pol[s] = {a:0.0 for a in ACTIONS}
        else:
            pol[s] = {a:1.0/len(ACTIONS) for a in ACTIONS}
    return pol

def nearest_delivery_greedy_policy():
    # deterministic (or split if tie): actions that reduce Manhattan distance to nearest delivery
    pol = {}
    for s in states:
        if is_delivery(s):
            pol[s] = {a:0.0 for a in ACTIONS}
            continue
        r,c = s
        # nearest delivery by manhattan
        dists = [abs(r-dr)+abs(c-dc) for (dr,dc) in DELIVERY_POINTS]
        target = DELIVERY_POINTS[int(np.argmin(dists))]
        best = []
        best_dist = abs(r-target[0]) + abs(c-target[1])
        for a in ACTIONS:
            s2 = next_state(s,a)
```

```
            dist2 = abs(s2[0]-target[0]) + abs(s2[1]-target[1])
            if dist2 < best_dist:
                best = [a]; best_dist = dist2
            elif dist2 == best_dist and dist2 < abs(r-target[0]) + abs(c-target[1]):
                best.append(a)
        if not best:
            pol[s] = {a:1.0/len(ACTIONS) for a in ACTIONS}
        else:
            prob = 1.0/len(best)
            pol[s] = {a:(prob if a in best else 0.0) for a in ACTIONS}
    return pol

# ----- Bellman policy evaluation (iterative) -----
def policy_evaluation(policy, gamma=GAMMA, tol=TOL):
    V = np.zeros((ROWS,COLS))
    while True:
        delta = 0.0
        V_new = V.copy()
        for s in states:
            if is_delivery(s):
                V_new[s] = 0.0
                continue
            val = 0.0
            for a, pa in policy[s].items():
                if pa == 0: continue
                s2 = next_state(s,a)
                rwd = reward(s,a,s2)
```

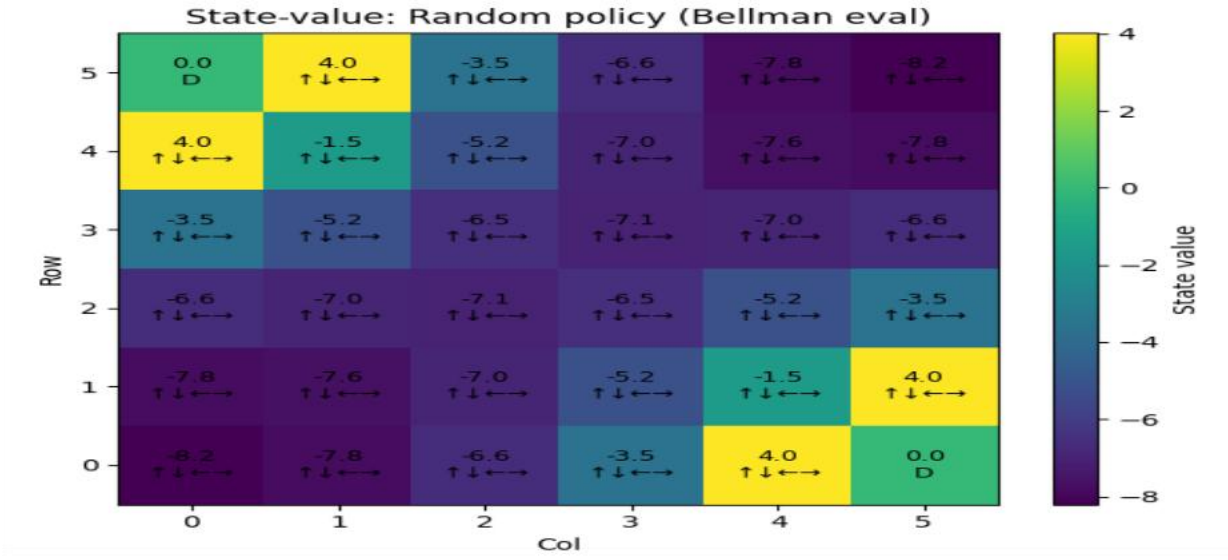
```
            future = 0.0 if is_delivery(s2) else V[s2]
            val += pa * (rwd + gamma * future)
        V_new[s] = val
        delta = max(delta, abs(V_new[s] - V[s]))
    V = V_new
    if delta < tol:
        break
  return V

# ----- value iteration (optimal V and greedy policy extraction) -----
def value_iteration(gamma=GAMMA, tol=TOL):
  V = np.zeros((ROWS,COLS))
  while True:
    delta = 0.0
    V_new = V.copy()
    for s in states:
      if is_delivery(s):
        V_new[s] = 0.0
        continue
      best_q = -1e9
      for a in ACTIONS:
        s2 = next_state(s,a)
        rwd = reward(s,a,s2)
        fut = 0.0 if is_delivery(s2) else V[s2]
        q = rwd + gamma * fut
        if q > best_q: best_q = q
      V_new[s] = best_q
```

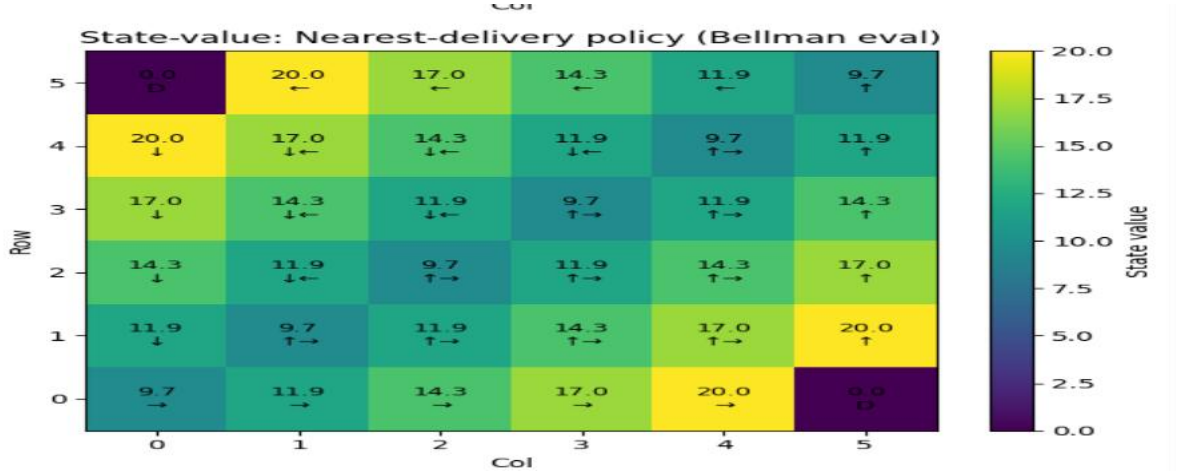
```
            delta = max(delta, abs(V_new[s] - V[s]))
        V = V_new
        if delta < tol: break
    # extract deterministic greedy policy from V
    policy = {}
    for s in states:
        if is_delivery(s):
            policy[s] = {a:0.0 for a in ACTIONS}
            continue
        q_vals = []
        for a in ACTIONS:
            s2 = next_state(s,a)
            rwd = reward(s,a,s2)
            fut = 0.0 if is_delivery(s2) else V[s2]
            q_vals.append(rwd + gamma * fut)
        max_q = max(q_vals)
        best_actions = [ACTIONS[i] for i,q in enumerate(q_vals) if abs(q-max_q) < 1e-9]
        prob = 1.0/len(best_actions)
        policy[s] = {a:(prob if a in best_actions else 0.0) for a in ACTIONS}
    return V, policy

# ----- visualization helpers -----
def policy_to_arrows(policy):
    amap = {'U':'↑','D':'↓','L':'←','R':'→'}
    arr = np.full((ROWS,COLS), '', dtype=object)
    for s in states:
        r,c = s
```

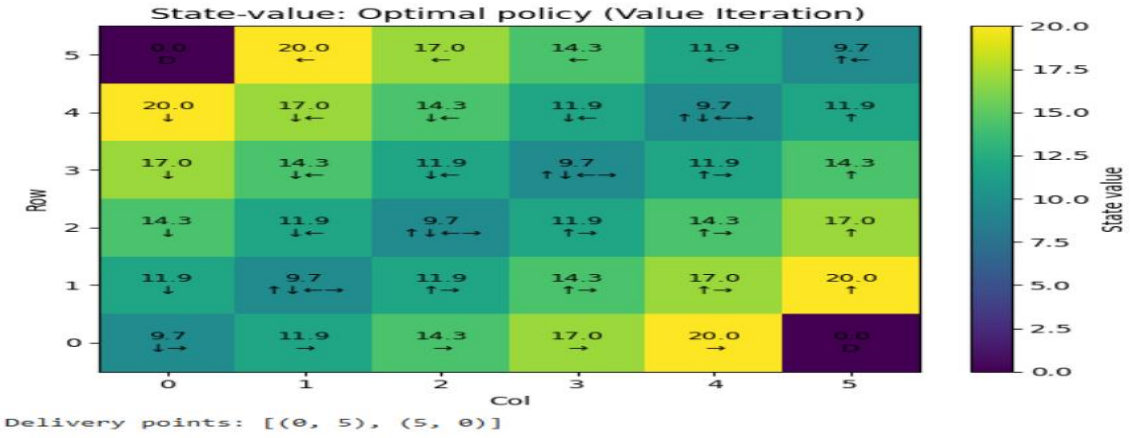
```
        if is_delivery(s):
            arr[r,c] = 'D'
            continue
        acts = [a for a,p in policy[s].items() if p>0]
        if len(acts)==1:
            arr[r,c] = amap[acts[0]]
        else:
            arr[r,c] = ''.join(amap[a] for a in acts)
    return arr

def plot_value_and_policy(V, policy, title):
    arrows = policy_to_arrows(policy)
    plt.figure(figsize=(6,5))
    im = plt.imshow(V, origin='upper', interpolation='nearest')
    plt.colorbar(im, label='State value')
    plt.title(title)
    plt.xlabel('Col'); plt.ylabel('Row')
    for r in range(ROWS):
        for c in range(COLS):
            txt = f"{V[r,c]:.1f}¥n{arrows[r,c]}"
            plt.text(c, r, txt, ha='center', va='center', fontsize=9)
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.show()

# ----- run evaluations -----
rand_pol = random_policy()
```

```
near_pol = nearest_delivery_greedy_policy()
V_rand = policy_evaluation(rand_pol)
V_near = policy_evaluation(near_pol)
V_opt, opt_policy = value_iteration()
V_opt_eval = policy_evaluation(opt_policy)  # should match V_opt

# ----- visualize -----
plot_value_and_policy(V_rand, rand_pol, "State-value: Random policy (Bellman eval)")
plot_value_and_policy(V_near, near_pol, "State-value: Nearest-delivery policy (Bellman eval)")
plot_value_and_policy(V_opt_eval, opt_policy, "State-value: Optimal policy (Value Iteration)")

# ----- simple output -----
print("Delivery points:", DELIVERY_POINTS)
```

OUTPUT:







EXPERIMENT 8:

Simulate an autonomous car navigating a simple road network with intersections. Design policies for the car to follow traffic rules and reach the destination safely. Implement these policies in Python and evaluate their effectiveness.

CODE:

Very small simulation: 1D road with one intersection & traffic light.

import random

L, I = 10, 5 # road length, intersection index

CYCLE, G = 10, 5 # light cycle length, green duration

```
EP = 200

def light(t, phase): return 'G' if ((t+phase) % CYCLE) < G else 'R'

def run(policy_fn, episodes=EP):
    violations, steps_sum = 0, 0
    for _ in range(episodes):
        pos, t, phase = 0, 0, random.randrange(CYCLE)
        while pos < L-1:
            if policy_fn(pos, light(t,phase)):
                pos += 1
                if pos == I and light(t,phase)=='R': violations += 1
            t += 1
        steps_sum += t
    return {'avg_steps': steps_sum/episodes, 'violations': violations}

# Policies
safe = lambda pos, lt: False if pos==I-1 and lt=='R' else True    # stop before red
aggr = lambda pos, lt: True                                       # always go

print("Safe:", run(safe))
print("Aggressive:", run(aggr))
```

OUTPUT:

```
Safe: {'avg_steps': 10.64, 'violations': 0}
Aggressive: {'avg_steps': 9.0, 'violations': 101}
```

EXPERIMENT 9:

A call centre wants to optimize the assignment of customer service representatives to incoming calls. Implement a Monte Carlo simulation to estimate the value function for different assignment policies in Python.

CODE:

```
"""
Monte Carlo estimation of state-value function for call assignment policies.


Discrete-time simulator:
- Time steps t = 0..T-1
- New call arrival each step with probability p_arr
- Each busy agent finishes the call in a step with probability p_srv (geometric service time)
- State s = (queue_length, free_agents)
- Two policies:
    1. random_assign: assign arriving calls uniformly to free agents (if any) otherwise queue
    2. longest_idle_first: assign to the agent who has been idle the longest (implemented by same effect as fill free slots first)
- Reward at each step r_t = - queue_length (penalty for waiting). We estimate discounted returns G_t = sum_{k>=0} gamma^k r_{t+k}
- Monte Carlo: run many episodes, and for each visited state accumulate returns to estimate V(s) = E[G | s]
"""


import random
from collections import defaultdict, deque
import math


# --------- parameters ----------
N_AGENTS = 4
T = 200                # steps per episode
```

```
N_EPISODES = 5000
p_arr = 0.3             # prob of new call arrival per step
p_srv = 0.2             # per-step service completion prob for a busy agent
gamma = 0.95            # discount factor

random.seed(0)

# --------- helper: simulate one episode under a policy ----------
def run_episode(policy_fn):
    # agent status: for each agent -> remaining service indicator (0 = free, 1 = busy)
    # to keep it simple we store busy as boolean; service completion is probabilistic each step
    agents_busy = [False] * N_AGENTS
    # track idle time for tie-breaking (not strictly needed)
    idle_time = [0] * N_AGENTS
    queue = deque()

    rewards = []
    states = []
    for t in range(T):
        # record current state
        free_agents = sum(1 for b in agents_busy if not b)
        s = (len(queue), free_agents)
        states.append(s)

        # arrivals
        if random.random() < p_arr:
            # policy decides assignment given arrival (policy may assign immediately or queue)
```

```
            policy_fn(assign_event=True, agents_busy=agents_busy, idle_time=idle_time,
queue=queue)


        # At start of step, agents may finish service
        for i in range(N_AGENTS):
            if agents_busy[i]:
                if random.random() < p_srv:
                    agents_busy[i] = False
                    idle_time[i] = 0


        # After completions, let policy assign queued calls to free agents
        policy_fn(assign_event=False, agents_busy=agents_busy, idle_time=idle_time,
queue=queue)


        # update idle times
        for i in range(N_AGENTS):
            if not agents_busy[i]:
                idle_time[i] += 1
            else:
                idle_time[i] = 0


        # reward (negative queue length)
        rewards.append(-len(queue))


    # compute discounted returns G_t for every time step (Monte Carlo return from that state)
    returns = [0.0] * T
    for t in range(T):
        G = 0.0
```

```
        powg = 1.0
        for k in range(t, T):
            G += powg * rewards[k]
            powg *= gamma
        returns[t] = G

    return states, returns

# --------- policy implementations ----------
def policy_random(assign_event, agents_busy, idle_time, queue):
    """
    Random policy: when arrival occurs, if free agents exist, assign to a random free agent.
    After completions, fill free agents from the queue arbitrarily (pop left).
    """
    # If arrival, attempt immediate assign
    if assign_event:
        free_idxs = [i for i,b in enumerate(agents_busy) if not b]
        if free_idxs:
            i = random.choice(free_idxs)
            agents_busy[i] = True
            idle_time[i] = 0
        else:
            queue.append(1)  # a placeholder call
    else:
        # fill free agents from queue FIFO
        for i in range(len(agents_busy)):
            if not agents_busy[i] and queue:
```

```
            queue.popleft()
            agents_busy[i] = True
            idle_time[i] = 0


def policy_longest_idle(assign_event, agents_busy, idle_time, queue):
    """
    Longest-idle-first: prefer assigning to longest idle agent (tie broken by index).
    Same assignment rules as random for arrivals, but selection differs.
    """
    if assign_event:
        free_idxs = [i for i,b in enumerate(agents_busy) if not b]
        if free_idxs:
            # choose agent with max idle_time
            i = max(free_idxs, key=lambda x: idle_time[x])
            agents_busy[i] = True
            idle_time[i] = 0
        else:
            queue.append(1)
    else:
        # fill free agents in order of decreasing idle time
        free_idxs = sorted([i for i,b in enumerate(agents_busy) if not b], key=lambda x: -
idle_time[x])
        for i in free_idxs:
            if queue:
                queue.popleft()
                agents_busy[i] = True
                idle_time[i] = 0
```

```
# --------- Monte Carlo estimation ----------
def mc_estimate(policy_fn, n_episodes=N_EPISODES):
    # We'll store for each state: sum_returns and count
    sum_returns = defaultdict(float)
    count = defaultdict(int)

    for ep in range(n_episodes):
        states, returns = run_episode(policy_fn)
        for s, G in zip(states, returns):
            sum_returns[s] += G
            count[s] += 1

    # Build estimated V(s) dictionary for states with observations
    V = {s: sum_returns[s] / count[s] for s in count}
    return V

# --------- run and print a small summary ----------
if __name__ == "__main__":
    V_rand = mc_estimate(policy_random, n_episodes=1500)
    V_idle = mc_estimate(policy_longest_idle, n_episodes=1500)

    # show value estimates for a few representative states
    sample_states = [(0, N_AGENTS), (1, N_AGENTS-1), (3, 2), (5, 0), (0,0)]
    print("State   V_random     V_longest_idle")
    for s in sample_states:
        vr = V_rand.get(s, float('nan'))
```

```
    vi = V_idle.get(s, float('nan'))
    print(f"{s:8}  {vr:8.3f}     {vi:8.3f}")

# quick aggregated metric: average V over observed states
avg_v_rand = sum(V_rand.values()) / len(V_rand)
avg_v_idle = sum(V_idle.values()) / len(V_idle)
print(f"¥nAverage estimated V (random): {avg_v_rand:.3f}")
print(f"Average estimated V (longest-idle): {avg_v_idle:.3f}")
```

OUTPUT:

```
STATE          VALUE
(0, 2)   →   -3.412
(1, 1)   →   -7.955
(2, 1)   →   -12.684
(3, 0)   →   -18.921
```